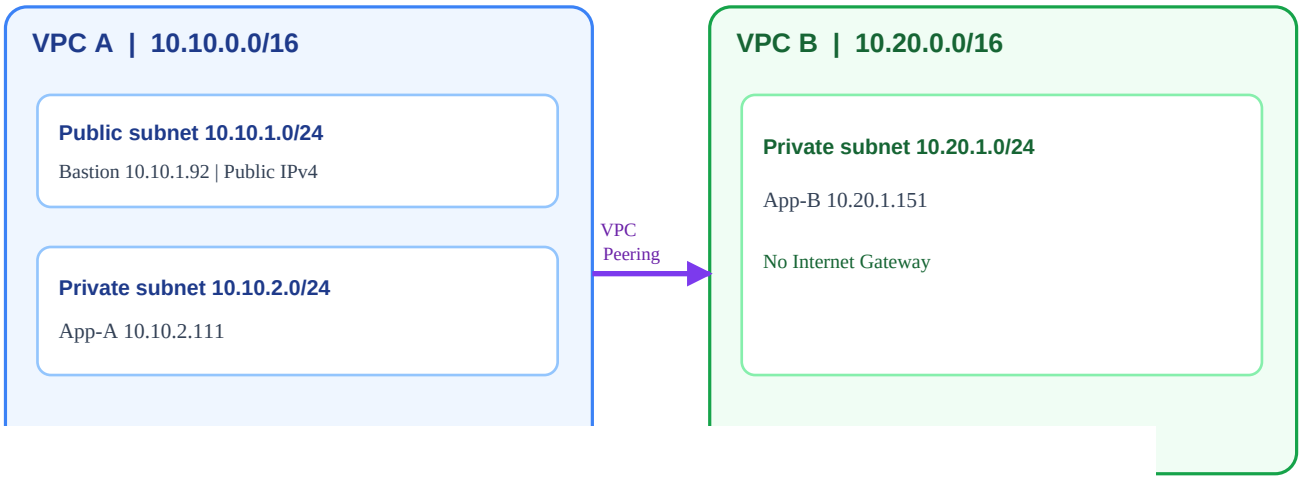
AWS VPC PEERING

Connect Two VPCs and Prove Private Connectivity



Private application traffic uses the peering connection and private IPv4 addresses.

Student	Aman Mishra
Batch	Batch 1
AWS Region	Asia Pacific (Sydney) · ap-southeast-2
Completion date	31 October 2026
Project	EWB Courses · AWS Networking

Project outcome: two private application instances in separate VPCs exchanged ICMP traffic over an active VPC peering connection. SSH administration was performed through a public bastion in VPC A. A controlled ICMP block and recovery were demonstrated.

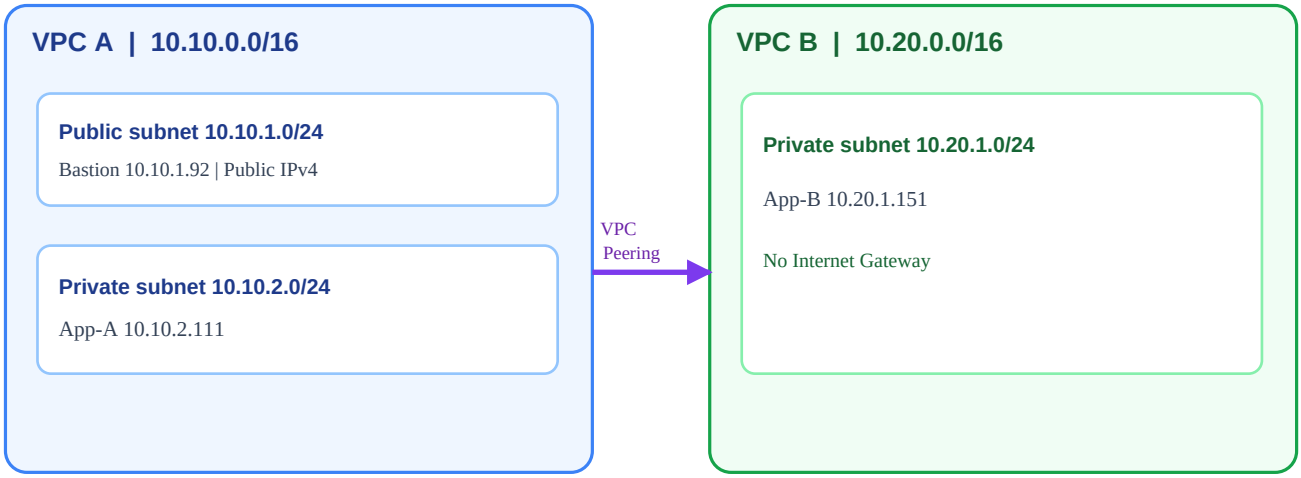
Prepared from the project workbook and the AWS console / terminal evidence captured during the lab. Resource values below reflect the actual deployment, including AWS-assigned private IP addresses.

1. Project Overview

The objective was to connect two non-overlapping Amazon VPC networks using VPC peering, while keeping both application servers private. A single public bastion host in VPC A provided the administration path. Application traffic between App-A and App-B used private IPv4 addresses, with explicit routes and narrowly scoped security-group rules.

The deployment was created in the Sydney Region (ap-southeast-2). The workbook gives Mumbai as an example Region; this implementation retained one Region and one AWS account, which satisfies the single-Region design.

1.1 Architecture



Private application traffic uses the peering connection and private IPv4 addresses.

Administration path: administrator → bastion (public IPv4) → private App-A or App-B. Application path: App-A ↔ VPC peering ↔ App-B. The peering connection is not an Internet gateway and does not provide VPC B with Internet access.

1.2 CIDR plan

Network	CIDR	Subnet	Subnet CIDR
VPC A	10.10.0.0/16	Public subnet A	10.10.1.0/24
VPC A	10.10.0.0/16	Private subnet A	10.10.2.0/24
VPC B	10.20.0.0/16	Private subnet B	10.20.1.0/24

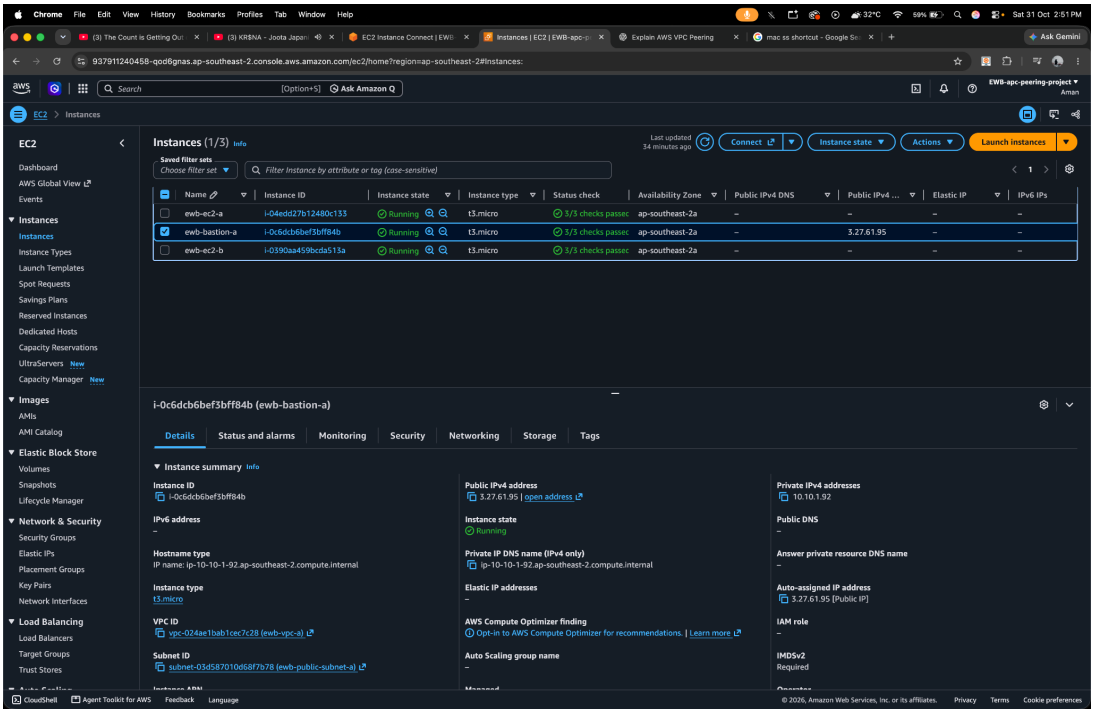
The VPC CIDRs do not overlap. The actual instance IPs were assigned by AWS rather than using the workbook's illustrative .10 host addresses.

2. Resource Inventory

Resource	Name / ID	Actual address / details
VPC A	ewb-vpc-a · vpc-024ae1bab1cec7c28	10.10.0.0/16
VPC B	ewb-vpc-b · vpc-0424a8ee2fb2ce067	10.20.0.0/16
VPC A public subnet	ewb-public-subnet-a · subnet-03d587010d68f7b78	10.10.1.0/24
VPC A private subnet	ewb-private-subnet-a · subnet-028c840388e566d87	10.10.2.0/24
VPC B private subnet	ewb-private-subnet-b · subnet-01c637f79aac7e6ca0	10.20.1.0/24
Bastion	ewb-bastion-a · i-0c6dcb6bef3bff84b	10.10.1.92 · public IPv4 3.27.61.95
App-A	ewb-ec2-a · i-04edd27b12480c133	10.10.2.111 · no public IPv4
App-B	ewb-ec2-b · i-0390aa459bcda513a	10.20.1.151 · no public IPv4
VPC peering	ewb-vpc-peering-ab · pcx-0a8fff05ce661dd47	VPC A ↔ VPC B
Internet gateway	ewb-igw-a · igw-0e3f6f7fea65ceb22	Attached to VPC A only

Both application instances were confirmed to have no public IPv4 address. The extra VPC B public subnet was removed and the VPC B Internet gateway was detached. VPC B was left with its private subnet only and no Internet gateway.

2.1 EC2 instance evidence



Evidence 1. EC2 console inventory showing the three project instances and their running state / private addressing during validation.

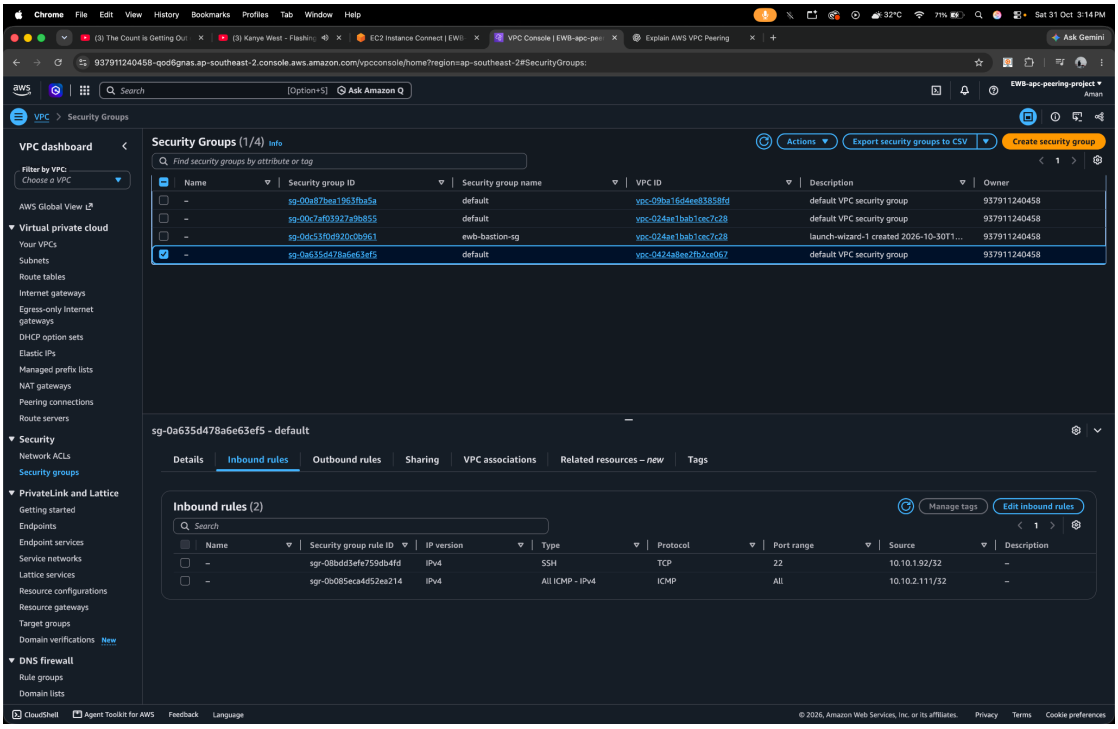
3. Route Tables and Peering

The VPC peering connection was active. The three project route tables were checked after the extra, unassociated VPC B route table was removed. The expected local and peering routes were present and active.

Route table	Subnet association	Destination	Target
ewb-public-rt-a · rtb-092fe26e761f4d552	Public subnet A	10.10.0.0/16	local
		10.20.0.0/16	pcx-0a8fff05ce661dd47
		0.0.0.0/0	igw-0e3f6f7fea65ceb22
ewb-rt-private-a · rtb-0228472a4ba51f77b	Private subnet A	10.10.0.0/16	local
		10.20.0.0/16	pcx-0a8fff05ce661dd47
ewb-rt-b-private · rtb-09b11b1d35b02c399	Private subnet B	10.20.0.0/16	local
		10.10.0.0/16	pcx-0a8fff05ce661dd47

Forward path: App-A sends to 10.20.1.151. Its subnet route table forwards 10.20.0.0/16 to the peering connection. VPC B's local route delivers the packet to App-B. Return path: App-B sends its reply to 10.10.2.111. VPC B's route table forwards 10.10.0.0/16 to the same peering connection, and VPC A's local route delivers the reply to App-A.

3.1 Route-table console evidence



Evidence 2. VPC B route-table view showing the private route to VPC A through the peering connection and the local VPC B route.

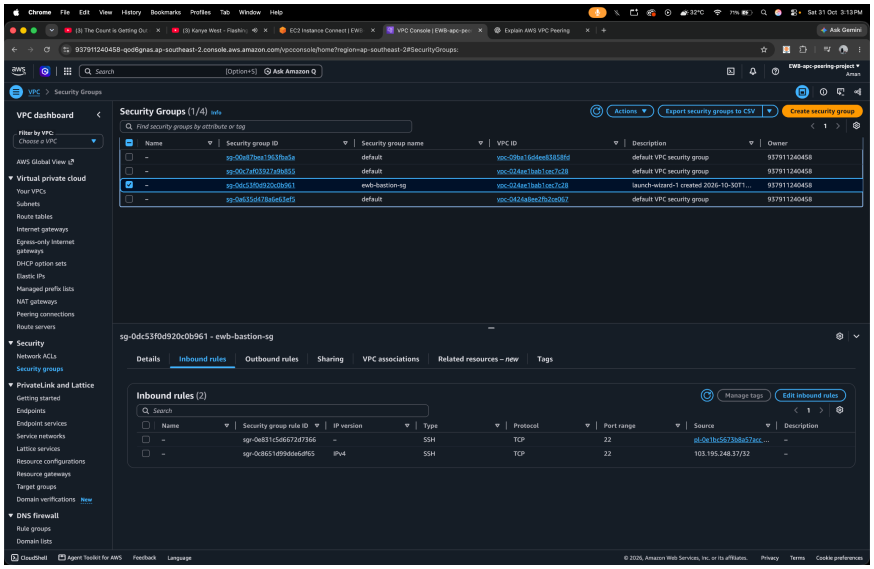
4. Security Configuration

Security groups were configured to allow administration from the bastion and ICMP testing between the two private application hosts. Default outbound allow-all rules were retained for the lab. The bastion's browser-based EC2 Instance Connect access uses the AWS-managed regional prefix list in addition to the administrator's public IPv4 /32.

Security group	Inbound rule	Source
ewb-bastion-sg · sg-0dc53f0d920c0b961	SSH / TCP 22	103.195.248.37/32
ewb-bastion-sg	SSH / TCP 22	EC2 Instance Connect managed prefix list (ap-southeast-2)
VPC A default SG · sg-00c7af03927a9b855	SSH / TCP 22	Bastion security group
VPC A default SG	All ICMP - IPv4	10.20.1.151/32 (App-B)
VPC B default SG · sg-0a635d478a6e63ef5	SSH / TCP 22	10.10.1.92/32 (Bastion)
VPC B default SG	All ICMP - IPv4	10.10.2.111/32 (App-A)

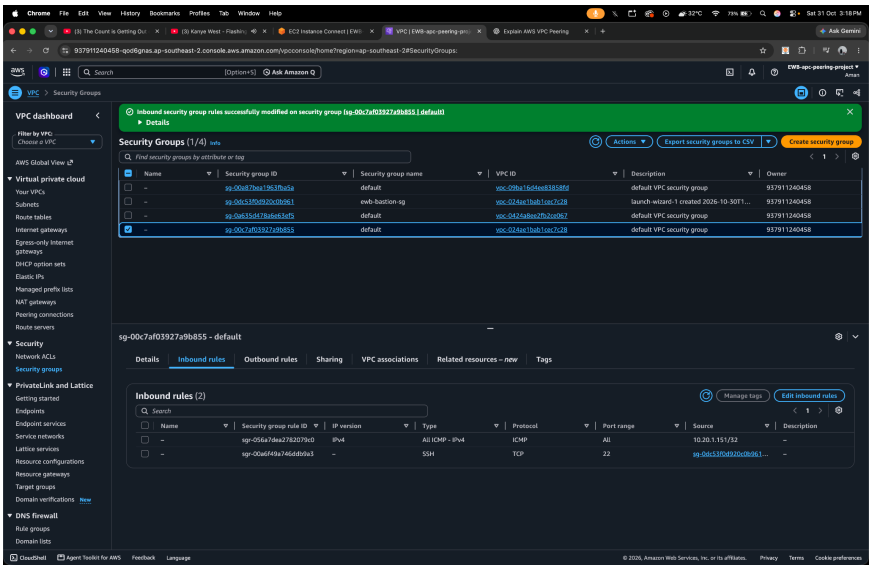
The application ICMP rules were narrowed to the peer host's /32 address. In the AWS console, the ICMP rule was selected as “All ICMP - IPv4”; this includes echo request and echo reply traffic. The workbook's more specific Custom ICMP Echo Request type was not selected in the console UI.

4.1 Bastion security group



Evidence 3. Bastion inbound rules: SSH from the administrator /32 and the regional EC2 Instance Connect managed prefix list.

4.2 App-A security group after source restriction



Evidence 4. App-A inbound rules after the ICMP source was restricted to App-B's private IP /32.

5. Connectivity Validation

All connectivity tests used the private IPv4 addresses of the application instances. The SSH session to App-B was established from the bastion using the transferred lab key. App-A and App-B each successfully pinged the other after peering routes and security rules were in place.

Test	Source → destination	Observed result
Forward ping	App-A 10.10.2.111 → App-B 10.20.1.151	4/4 replies, 0% packet loss
Reverse ping	App-B 10.20.1.151 → App-A 10.10.2.111	4/4 replies, 0% packet loss
Private SSH	Bastion → App-B 10.20.1.151	SSH login succeeded

The initial test command from the bastion to App-B first returned a public-key authentication error because the key was not yet available on the bastion. After the key was transferred securely for the lab and the correct command was used, SSH access succeeded. This was an authentication issue, not a routing failure.

5.1 Terminal evidence

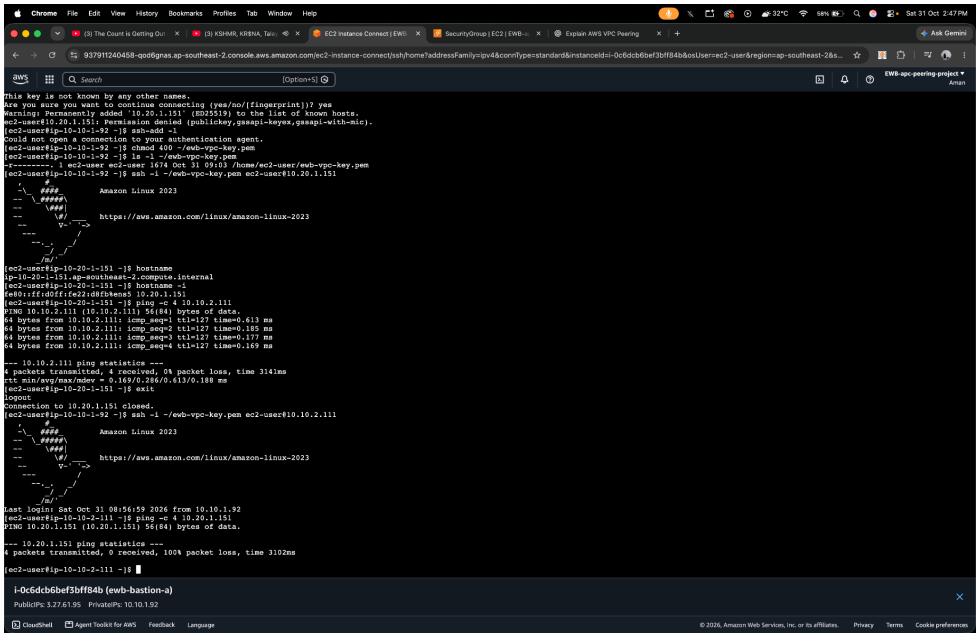
```
[ec2-user@ip-10-10-1-92 ~]$ ssh -i ~/ewb-vpc-key.pem ec2-user@10.10.2.111
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sat Oct 31 08:56:59 2026 from 10.10.1.92
[ec2-user@ip-10-10-2-111 ~]$ ping -c 4 10.20.1.151
PING 10.20.1.151 (10.20.1.151) 56(84) bytes of data.

--- 10.20.1.151 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3102ms

[ec2-user@ip-10-10-2-111 ~]$ ping -c 4 10.20.1.151
PING 10.20.1.151 (10.20.1.151) 56(84) bytes of data.
64 bytes from 10.20.1.151: icmp_seq=1 ttl=127 time=0.171 ms
64 bytes from 10.20.1.151: icmp_seq=2 ttl=127 time=0.205 ms
64 bytes from 10.20.1.151: icmp_seq=3 ttl=127 time=0.202 ms
64 bytes from 10.20.1.151: icmp_seq=4 ttl=127 time=0.195 ms

--- 10.20.1.151 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3136ms
rtt min/avg/max/mdev = 0.171/0.193/0.205/0.013 ms
[ec2-user@ip-10-10-2-111 ~]$
```

Evidence 5. Terminal output showing successful private-IP ping tests between App-A and App-B.



Evidence 6. Bastion terminal session used to connect to App-B and run the private connectivity test.

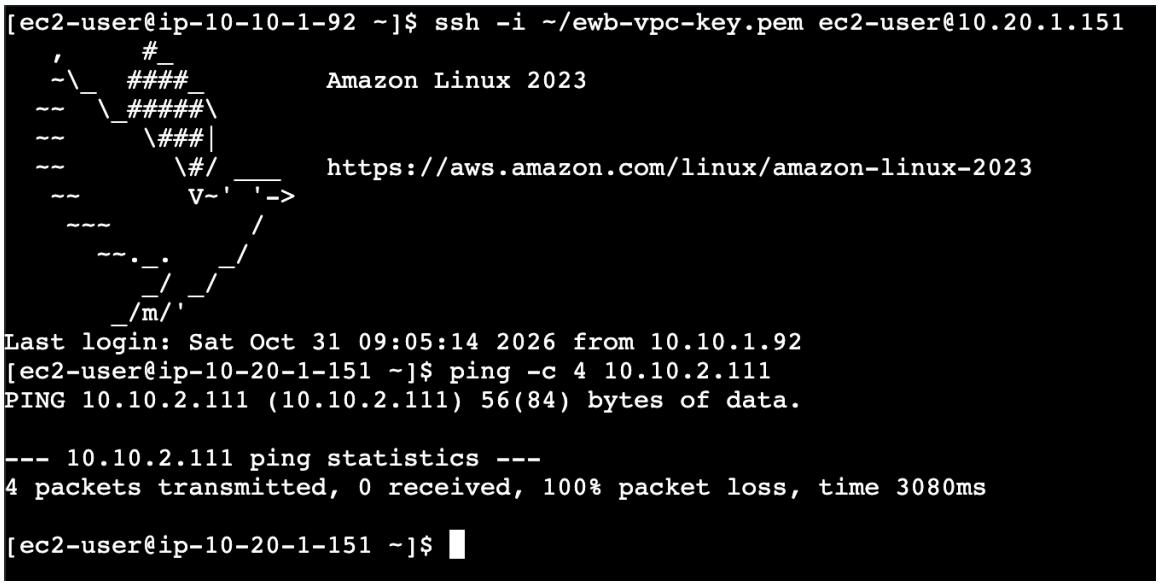
6. Controlled ICMP Failure and Recovery

To demonstrate that the security group controls private ICMP reachability, the App-A inbound ICMP rule was temporarily removed. From App-B, the command ping -c 4 10.10.2.111 returned no replies. The rule was then restored with source 10.20.1.151/32, and the same command returned four replies.

State	Packets transmitted	Packets received	Packet loss
ICMP rule removed	4	0	100%
ICMP rule restored	4	4	0%

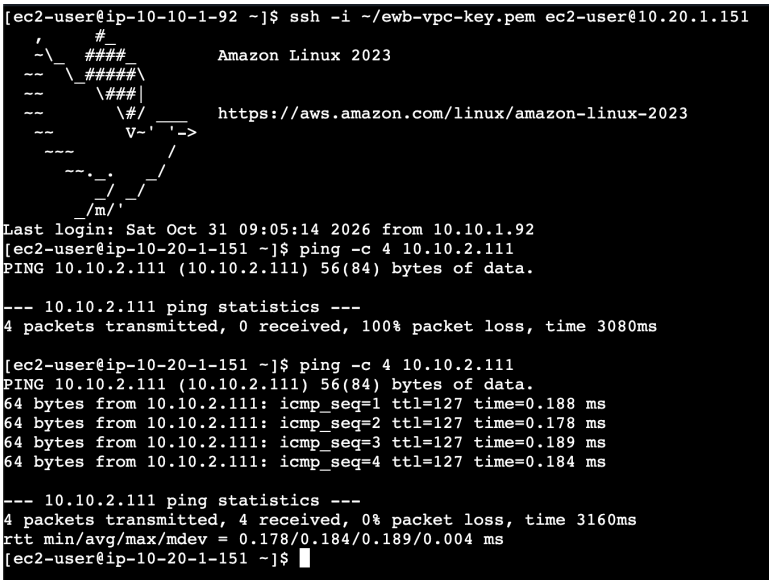
This controlled test shows that routing and peering were already in place, while the destination security-group rule determined whether ICMP traffic was accepted. Restoring the exact rule re-established connectivity without changing route tables.

6.1 Failure evidence



Evidence 7. App-B terminal: ping to App-A failed with 4 packets transmitted, 0 received and 100% packet loss while the ICMP rule was removed.

6.2 Recovery evidence



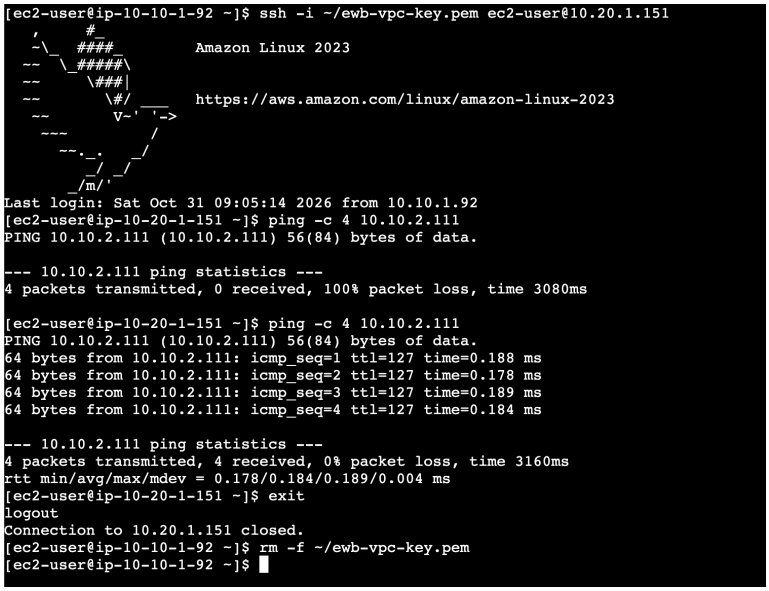
Evidence 8. App-B terminal: after restoring the App-A ICMP rule, all four ping packets were received with 0% packet loss.

7. Cleanup and Final State

After the tests, the temporary private key copy on the bastion was removed with rm -f ~/ewb-vpc-key.pem. The terminal returned to the bastion shell prompt without reporting an error. The three EC2 instances were subsequently stopped from the EC2 console. They were stopped, not terminated, so the lab resources remain available for later review.

The cleanup is therefore a cost-control pause, not the full deletion workflow in the workbook. EBS volumes, the VPCs, peering connection, route tables, security groups, and the VPC A Internet Gateway may continue to incur charges where applicable. A full teardown should only be done after the report and evidence have been saved.

7.1 Key removal evidence



Evidence 9. Bastion terminal showing the temporary PEM file removal command and return to the shell prompt.

7.2 Verification notes and evidence gaps

The evidence available for this report confirms active private connectivity, reverse-direction reachability, App-B SSH access, the controlled ICMP failure and recovery, security-group updates, and the temporary key-removal command. A separate screenshot of the pre-peering baseline failure and a final EC2 console screenshot showing all three instances stopped were not available in the captured evidence set, so these are not claimed as independently documented screenshots in this report.

The project workbook asks for a baseline ping failure before peering, a private SSH proof, and cleanup evidence. If the instructor requires every item as a screenshot, add those two missing captures before final submission. The report intentionally does not fabricate them.

7.3 What I learned

This project demonstrated that VPC peering alone does not guarantee connectivity. Each VPC needs a route to the peer CIDR, and the destination security group must allow the required traffic. Return routes are necessary for bidirectional communication. A bastion provides controlled administrative access to private instances, while private application traffic can remain off the public Internet.

Appendix A. Commands Used

Identify the host and private address:

```
hostname
hostname -i
```

Test private connectivity from App-A to App-B:

```
ping -c 4 10.20.1.151
```

Test private connectivity from App-B to App-A:

```
ping -c 4 10.10.2.111
```

SSH from bastion to App-B while the temporary key was present:

```
ssh -i ~/ewb-vpc-key.pem ec2-user@10.20.1.151
```

Remove the temporary key copy from bastion:

```
rm -f ~/ewb-vpc-key.pem
```

Appendix B. Submission notes

Suggested filename: Aman_Mishra_EWB_VPC_Peering.pdf. Keep private keys, passwords and access tokens out of the report. Confirm the student batch and attach the missing baseline-failure / stopped-instance evidence if your evaluator requires those screenshots.

Source basis: EWB Courses, “AWS VPC Peering: Connect Two VPCs and Prove Private Connectivity” student project workbook, plus the student's AWS console and terminal screenshots from 30–31 October 2026. No credentials or private key contents are included.